

Trabalhos 1, 2 e 4 - Comunicação Digital

Pedro Yochinori Gushiken

10 de dezembro de 2015

Sumário

1	Trabalho 1 - Comentários sobre ADPCM e Modulação Delta em comparação com DPCM e PCM	2
1.1	Modulação Delta	2
1.1.1	Comentários	2
1.1.2	Implementação	2
1.2	ADPCM	4
1.2.1	Comentários	4
1.3	Tabela Comparativa	4
1.4	Uso Comercial	5
2	Trabalho 2 - Código Comentado do Diagrama de Olho	6
2.1	Comentários sobre o Código	6
2.2	Degradações	8
2.2.1	Ruído Branco	8
2.3	Diagramas de olho	8
3	Trabalho 4 - Prática Computacional sobre BER e Pe em um canal AWGN	12
3.1	Código Comentado	12
3.2	Constelação	14
3.2.1	Sem Ruído	14
3.2.2	Com Ruído	16
3.3	Taxa de Erro de Bit e Probabilidade de Erro	18

Capítulo 1

Trabalho 1 - Comentários sobre ADPCM e Modulação Delta em comparação com DPCM e PCM

1.1 Modulação Delta

1.1.1 Comentários

Na modulação Delta, a mensagem é super amostrada (numa taxa muito superior a Nyquist), aumentando a correlação entre amostras adjacentes do sinal. Isto permite o uso de uma estratégia de quantização simples. Na forma mais básica de modulação Delta a diferença entre uma amostra e a outra é quantizada através de apenas um bit e a aproximação do sinal é construída fazendo um incremento de $\pm\Delta$, a depender da aproximação estar acima ou abaixo do nível do sinal amostrado (erro quantizado, multiplicado por delta e integrado discretamente ao longo de cada período). Quando o Δ usado é fixo esta estratégia é chamada de modulação Delta linear. Este esquema é chamado no Haykin de DPCM com preditor de nível 0.

A maior virtude da modulação Delta é sua simplicidade de implementação.

Quanto ao ruído de quantização, este esquema sofre de dois tipos:

- 1) "Sobrecarga de Inclinação"(slope overload), onde o Δ escolhido é pequeno demais para um determinado trecho do sinal onde a inclinação excede um certo valor e mesmo com o sinal do erro quantizado sendo constante num determinado intervalo a aproximação não é capaz de acompanhar o sinal. Este ruído também ocorre na modulação PCM diferencial.
- 2)"Ruído Granular"(Granular Noise), que é análogo ao ruído de quantização no PCM. É gerado quando a inclinação da reta é baixa. No caso de um Δ fixo, a aproximação sempre terá uma certa diferença em relação ao sinal amostrado. Isso pode ser claramente ilustrado caso se gere um sinal amostrado que é constante num determinado intervalo, neste caso o erro quantizado tenderá a uma sequência alternando constantemente entre $+1$ e -1 .

1.1.2 Implementação

Foi feita uma implementação simples de modulação Delta linear, o sinal amostrado contém quatro componentes senoidais com $\omega=200$ até 50 rad/s em incrementos de 50 radianos/s. O clock usado na amostragem é de 10kHz, que cumpre o requerimento de super amostragem. O filtro passa-baixa usado é de terceira ordem. O valor de Δ é de 0.05.

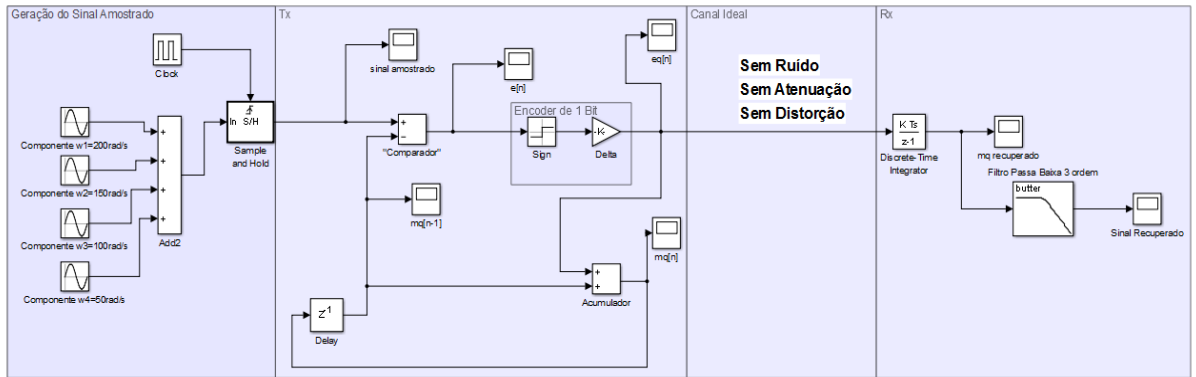


Figura 1.1: Esquema usado para implementação da Modulação Delta

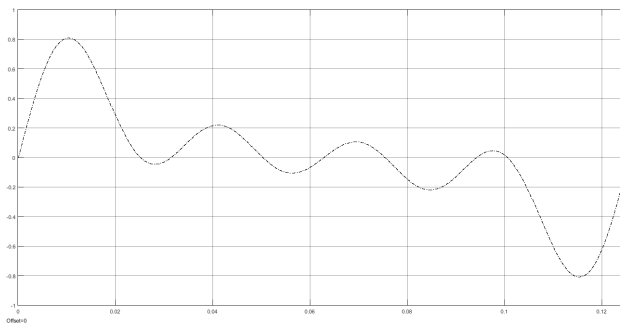


Figura 1.2: Sinal super amostrado

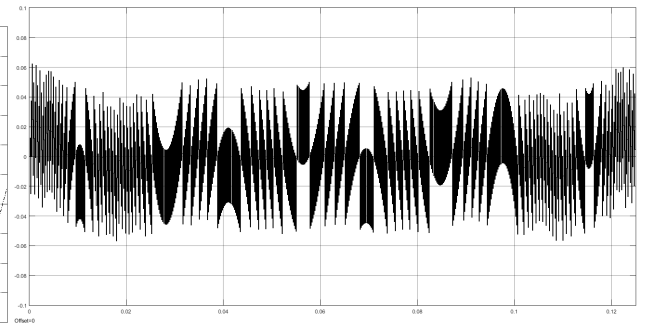


Figura 1.3: Erro $e[n]$ entre o sinal amostrado e a aproximação do sinal $m_q[n]$

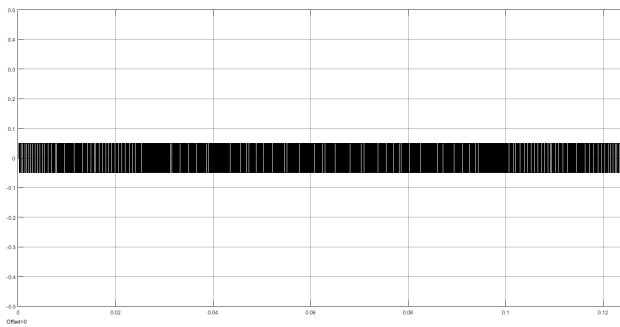


Figura 1.4: Erro $e[n]$ quantizado e multiplicado por Δ

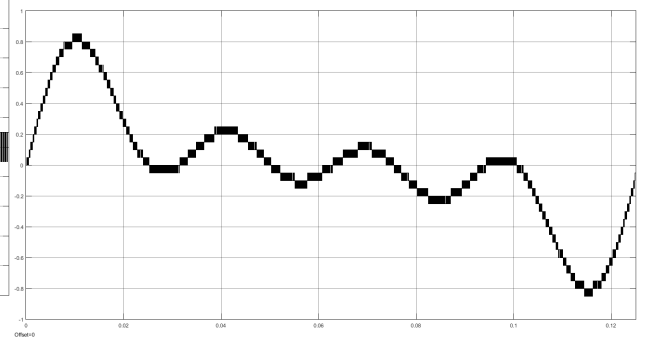


Figura 1.5: Aproximação do sinal $m_q[n]$

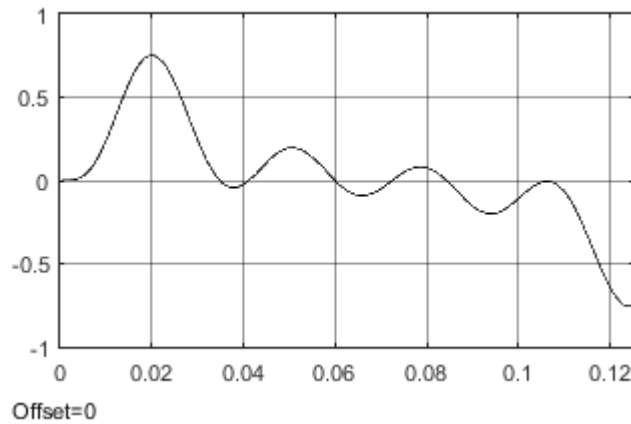


Figura 1.6: Sinal Recuperado

1.2 ADPCM

1.2.1 Comentários

O uso do PCM tradicional à uma taxa de transmissão de 64kbps para voz em muitas aplicações se torna muito custoso do ponto de vista do consumo de banda. Neste tipo de aplicação se torna necessário a codificação para atingir taxas de bit menores enquanto são mantidas a fidelidade e qualidade do áudio. Se torna necessário então o uso de um codificador de forma de onda, cuja filosofia de design tem em mente dois objetivos claros:

- 1) Remoção de redundâncias na representação do sinal.
- 2) Assinalar os bits disponíveis para as partes não redundantes do sinal de uma forma eficiente.

No caso do ADPCM, é possível reduzir a taxa de bits necessária para manter a mesma qualidade para 32kbps, através do uso de predição e quantização adaptativas. A modulação PCM Diferencial Adaptativa se aproveita do fato de que os sinais transmitidos geralmente tem uma característica não estacionária, isto é, a variância do sinal e autocorrelação mudam de acordo com o tempo. Este fato torna possível implementar um valor de passo para a quantização que muda de acordo com o tempo e estimativa da variância do sinal, assim como alterar os coeficientes do preditor de forma a melhor se ajustar ao sinal.

1.3 Tabela Comparativa

Tabela 1.1: Tabela de comparação qualitativa dos esquemas de modulação estudados

Parâmetro	PCM	DPCM	ADPCM	Modulação Delta
Consumo de Banda	Médio	Baixo	Muito Baixo	Baixo
Complexidade de Implementação	Baixa	Média	Alta	Muito Baixa
Ruído de Quantização	Alto	Baixo	Muito Baixo	Médio

1.4 Uso Comercial

O ADPCM em 32kbps é aceito como um padrão internacional para transmissão de voz em conjunto com o PCM a 64kbps. Não foram encontradas aplicações comerciais da modulação Delta linear ou suas variantes na pesquisa (o que não quer dizer que não existam).

Capítulo 2

Trabalho 2 - Código Comentado do Diagrama de Olho

2.1 Comentários sobre o Código

```
%(pnrz.m)
%Geração de um pulso retangular de largura T
%Uso da função: pout=pnrz(T);
%definição da função
function pout=prect(T);
%pout é um vetor de 1's de tamanho T
pout = ones(1,T);
end

%(prz.m)
%Geração de um pulso retangular de largura T/2
%Uso da função: pout = prz(T);
%definição da função:
function pout=prz(T);
%pout é um vetor com T/4 zeros, seguido de T/2 1's, seguido de T/4 zeros
pout = [zeros(1,T/4) ones(1, T/2) zeros(1, T/4)];
end

%(psine.m)
%geração de um pulso senoidal de largura T
%definição da função:
function pout=psine(T)
%pout é um vetor igual ao seno de  $\pi \cdot (T \text{ de } 0 \text{ a } T-1)/T$ 
pout = sin(pi*[0:T-1]/T);
end

%(prcos.m)
%Uso: y=prcos(rollfac,lenght,T)
%definição da função:
function y=prcos(rollfac,lenght, T)
%rollfac = 0 a 1 é o fator de roll off
%lenght é a largura onesided do pulso no número de T
```

```

%length = 2T + 1;
%T é a taxa de sobreamostragem
%rcosfir gera um filtro de cosseno levantado de acordo com os argumentos
y=rcosfir(rollfac,length,T,1,'normal');
end

%(binary_eye.m)
%Geração e Plotagem de Diagramas de Olho
%limpar as variáveis e a tela atual de figura
clear;clf;
data = sign(randn(1,400)); % gerar 400 bits randomizados

%geração de 400 pontos de ruído com variância r
%r = 0.01;
%ruid = randn(1,400)*sqrt(r);
%data = data + ruid

Tau = 64; %definir o período de símbolo
% gerar trem de impulsos inserindo Tau-1 zeros entre cada bit
dataup = upsample(data, Tau)
yrz = conv(dataup, prz(Tau)); %Sinal polar RZ, conv é a operação convolução
yrz = yrz(1:end-Tau+1); %retirar a "cauda" do vetor yrz
ynrz = conv(dataup, pnrz(Tau)); %Sinal polar NRZ
ynrz = ynrz(1:end-Tau+1); %Retirar a "cauda" do vetor ynrz
ysine = conv(dataup,psine(Tau)); %Meia Senóide Polar
ysine = ysine(1:end-Tau+1); %Retirar a "cauda" do vetor ysine
Td=4; %definição de Td
yrcos=conv(dataup, prcos(0.5,Td,Tau));

%cosseno levantado truncado em 4 períodos com rolloff =0.5
yrcos=yrcos(2*Td*Tau:end-2*Td*Tau+1); %Gerando trem de pulsos

%geração do diagrama de olho RZ
eye1=eyediagram(yrz,2*Tau,Tau,Tau/2); title('RZ eye-diagram');

%geração do diagrama de olho NRZ
eye2=eyediagram(ynrz,2*Tau,Tau,Tau/2); title('NRZ eye-diagram');

%geração do diagrama de olho Meia-Senóide
eye3=eyediagram(ysine, 2*Tau, Tau, Tau/2); title('Half-Sine eye-diagram');

%geração do diagrama de olho Cosseno-Levantado
eye4=eyediagram(yrcos,2*Tau,Tau); title('Raised-Cosine eye-diagram');

%(Mary_eye.m)
%Geração de Diagrama de Olho e Plotagem (PAM Quaternário)
%limpar as variáveis e telas de figura
clear;clf;
%geração de 400 símbolos PAM quaternário (valores possíveis: -1,-3, 1, 3)
data = sign(randn(1,400)) +2*sign(randn(1,400));

```

```

%geração de 400 pontos de ruído com variância r
%r = 0.01;
%ruid = randn(1,400)*sqrt(r);
%data = data + ruid

%definição do período de símbolo
Tau=64;
% gerar trem de impulsos inserindo Tau-1 zeros entre cada bit
dataup = upsample(data, Tau)
yrz = conv(dataup, prz(Tau)); %Sinal polar RZ, conv é a operação convolução
yrz = yrz(1:end-Tau+1); %retirar a "cauda" do vetor yrz
ynrz = conv(dataup, prnz(Tau)); %Sinal polar NRZ
ynrz = ynrz(1:end-Tau+1); %Retirar a "cauda" do vetor ynrz
ysine = conv(dataup, psine(Tau)); %Meia Senóide Polar
ysine = ysine(1:end-Tau+1); %Retirar a "cauda" do vetor ysine
Td=4; %definição de Td
yrcos=conv(dataup, prcos(0.5,Td,Tau));

%cosceno levantado truncado em 4 períodos com rolloff =0.5
yrcos=yrcos(2*Td*Tau:end-2*Td*Tau+1); %Gerando trem de pulsos

%geração do diagama de olho RZ
eye1=eyediagram(yrz,2*Tau,Tau,Tau/2); title('RZ eye-diagram');

%geração do diagrama de olho NRZ
eye2=eyediagram(ynrz,2*Tau,Tau,Tau/2); title('NRZ eye-diagram');

%geração do diagrama de olho Meia-Senóide
eye3=eyediagram(ysine, 2*Tau, Tau, Tau/2); title('Half-Sine eye-diagram');

%geração do diagrama de olho Cosseno-Levantado
eye4=eyediagram(yrcos,2*Tau,Tau); title('Raised-Cosine eye-diagram');

```

2.2 Degradações

2.2.1 Ruído Branco

O ruído branco AWGN pode ser gerado simplesmente através da função `randn()`, que gera uma distribuição normal, podemos gerar um vetor $R = \text{randn}(1,400) * \sqrt{r}$, adicionando este vetor ao vetor de dados gerado podemos enxergar qual a contribuição do ruído no diagrama de olho para um determinado r , que é a variância, neste caso escolhemos $r=0.01$ de forma a não fechar completamente o diagrama, mas ainda mostrar um pouco do "fechamento".

2.3 Diagramas de olho

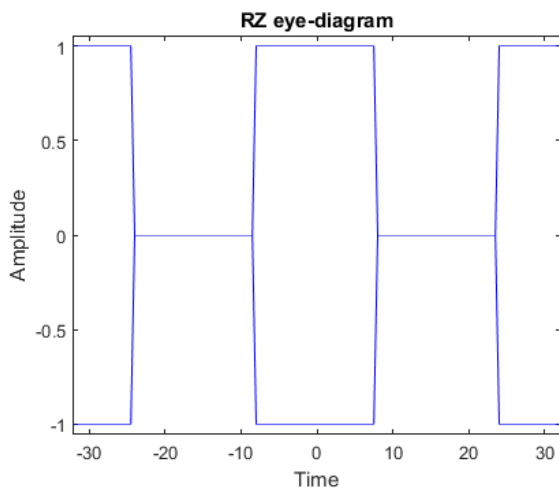


Figura 2.1: RZ Binário sem ruído

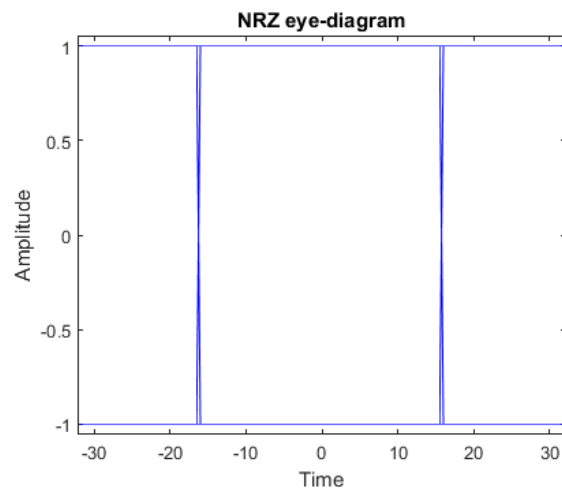


Figura 2.2: NRZ Binário sem ruído

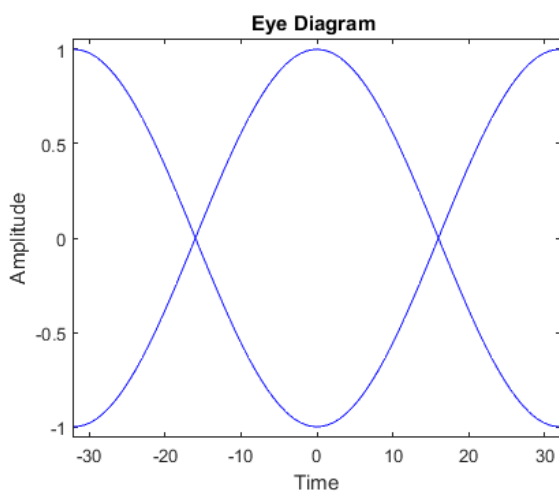


Figura 2.3: Meia Senóide Binário sem ruído

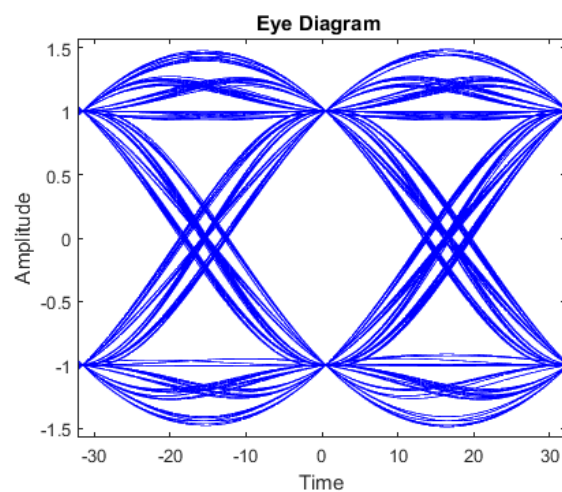


Figura 2.4: Cosseno Levantado Binário sem ruído

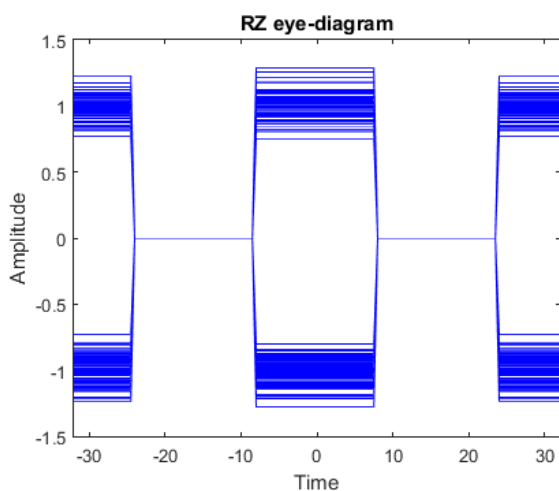


Figura 2.5: RZ Binário com ruído

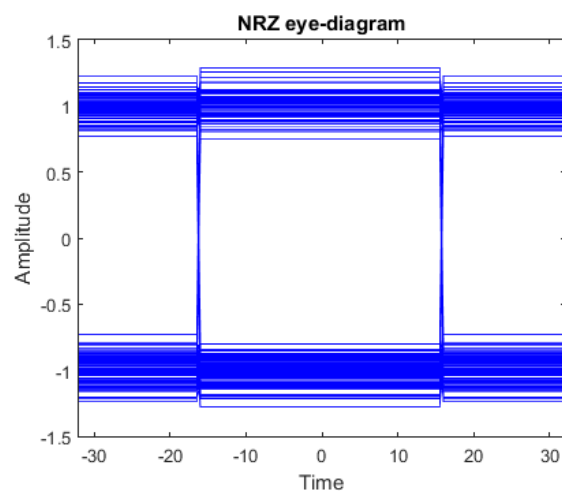


Figura 2.6: NRZ Binário com ruído

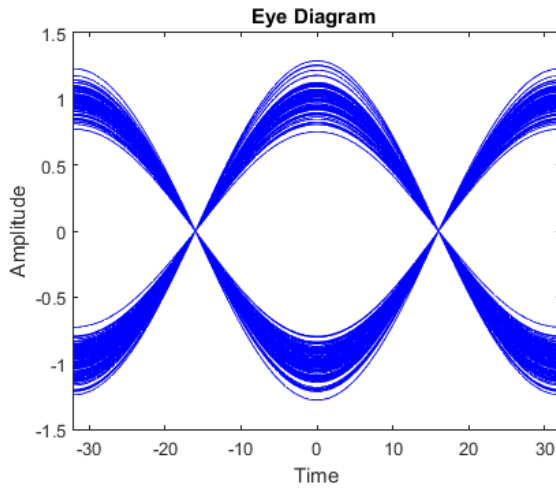


Figura 2.7: Meia Senóide Binário com ruído

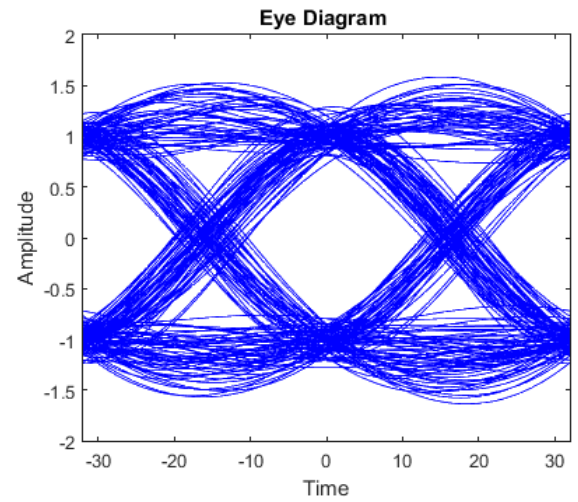


Figura 2.8: Cosseno Levantado Binário com ruído

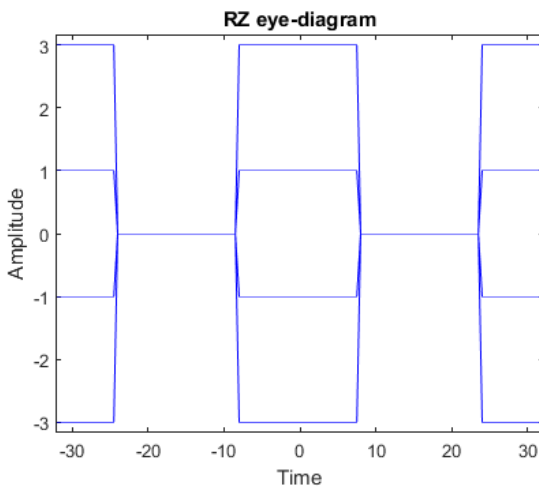


Figura 2.9: RZ Quaternário sem ruído

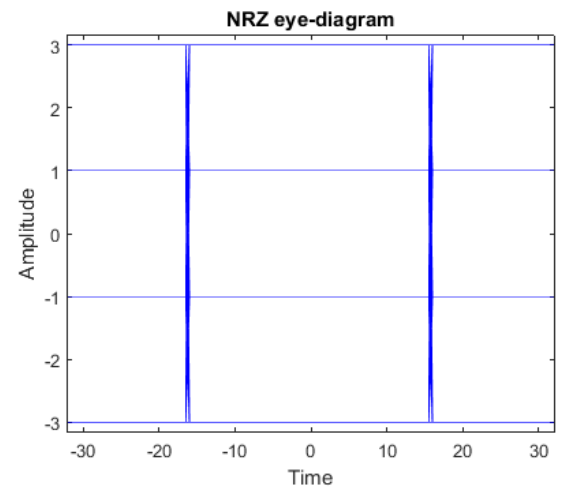


Figura 2.10: NRZ Quaternário sem ruído

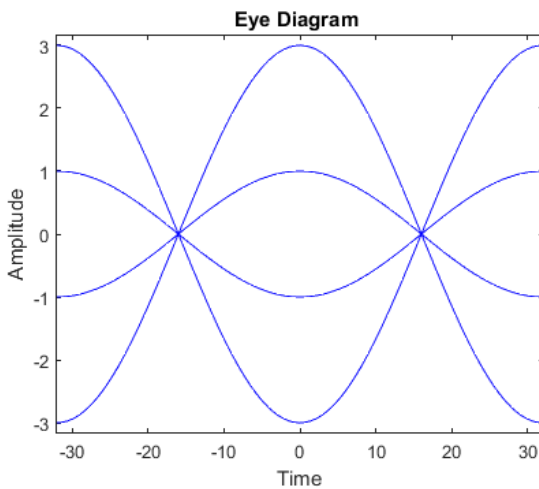


Figura 2.11: Meia Senóide Quaternário sem ruído

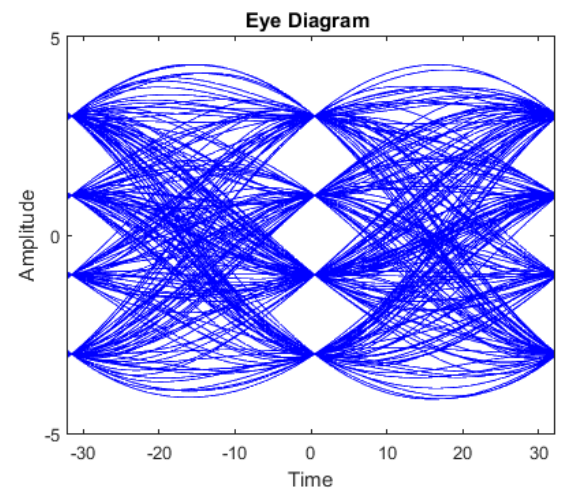


Figura 2.12: Cosseno Levantado Quaternário sem ruído

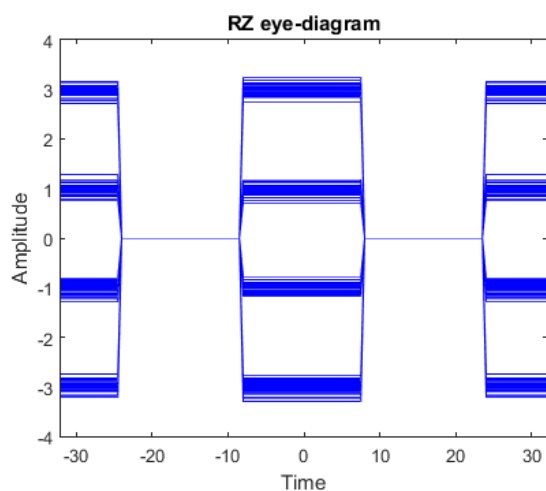


Figura 2.13: RZ Quaternário com ruído

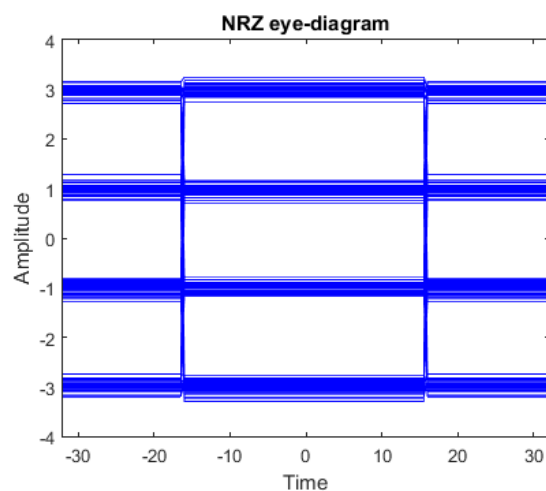


Figura 2.14: NRZ Quaternário com ruído

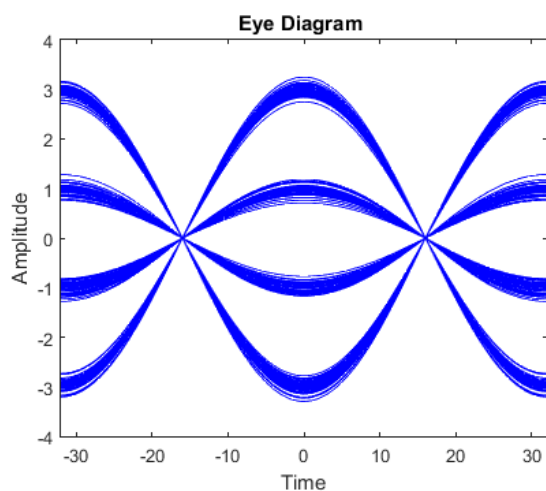


Figura 2.15: Meia Senóide Quaternário com ruído

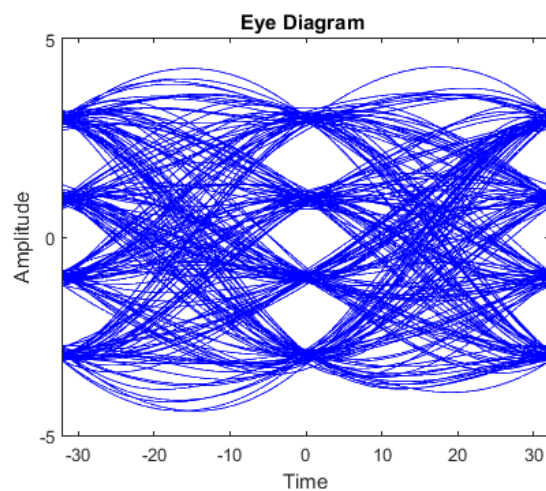


Figura 2.16: Cosseno Levantado Quaternário com ruído

Capítulo 3

Trabalho 4 - Prática Computacional sobre BER e Pe em um canal AWGN

3.1 Código Comentado

```
%declarar um objeto modulador PAM
hMod = comm.PAMModulator(2);
%declarar um objeto filtro cosseno levantado
hRCTxFilter = comm.RaisedCosineTransmitFilter(...
'Shape','Normal', ...
'RolloffFactor',0.5, ...
'FilterSpanInSymbols',Rup, ...
'OutputSamplesPerSymbol',Rup);
%taxa de upsampling
Rup = 64;
%número de bits = ndBit
% %para gerar os exemplos com 100 bits fazer ndBit=100 e comentar a parte do código
% %que gera as diferentes BERs
ndBit=100000;
%geração de dados
data = (ones(ndBit,1)+sign(rand(ndBit,1)-0.5))/2;
%convolução dos dados através da resposta do Modulador
modData = step(hMod, data);
%declarar objeto Diagrama de Constelação
hScope = comm.ConstellationDiagram('SamplesPerSymbol',Rup);
%convolução dos dados na saída do PAM através do filtro cosseno levantado
txSig = step(hRCTxFilter,modData);
%ganho casado
gain = 1/max(hRCTxFilter.coefs.Numerator);
%convolução dos dados na saída do PAM através do filtro casado
txSig = gain*step(hRCTxFilter,modData);
%Diagrama de Constelação sem Ruído
step(hScope,txSig);

%Geração do ruído AWGN
rcv = awgn(txSig,20,'measured');
%Diagrama de Constelação com Ruído
```

```

%step(hScope,rcv)
%EbNo, gerar valores de Eb/No entre 0.1 e 10 em incrementos de 0.1
EbNo=[0.1:100]/10;
%variável auxiliar de contagem de erros
c=0;
for a=1:100
%gerar novo ruído AWGN para EbNo(a)
    arcv=awgn(txSig,EbNo(a),'measured');
%trazer o ruído gerado para cada bit
    arcvd=downsample(arcv,Rup);
%recuperar os dados
    txSigD=downsample(txSig,Rup);
    for b=1:ndBit
        %lógica simples de contador
        if txSigD(b)>0
            if arcvd(b)<0
                c =c+1;
            end
        end
        if txSigD(b)<0
            if arcvd(b)>0
                c =c+1;
            end
        end
    end
    %cálculo da ber para 100000 pontos
    ber100k(a)=c/ndBit
    c=0;
end

%cálculo da BER para 10000 pontos, mesma lógica que o laço anterior
ndBit2 = 10000;
data2 = (ones(ndBit2,1)+sign(rand(ndBit2,1)-0.5))/2;
modData2 = step(hMod, data2);
gain2 = 1/max(hRCTxFilter.coeffs.Numerator);
txSig2 = gain2*step(hRCTxFilter,modData);
for a=1:100
    arcv=awgn(txSig2,EbNo(a),'measured');
    arcvd=downsample(arcv,Rup);
    txSigD=downsample(txSig2,Rup);
    for b=1:ndBit2
        if txSigD(b)>0
            if arcvd(b)<0
                c =c+1;
            end
        end
        if txSigD(b)<0
            if arcvd(b)>0
                c =c+1;
            end
        end
    end
end

```

```

        end
    end
end
ber10k(a)=c/ndBit2
c=0;
end

%cálculo da BER para 1000 pontos, mesma lógica dos outros laços
ndBit3 = 1000;
data3 = (ones(ndBit3,1)+sign(rand(ndBit3,1)-0.5))/2;
modData3 = step(hMod, data3);
gain3 = 1/max(hRCTxFilter.coeffs.Numerator);
txSig3 = gain3*step(hRCTxFilter,modData);
for a=1:100
    arcv=awgn(txSig3,EbNo(a),'measured');
    arcvd=downsample(arcv,Rup);
    txSigD=downsample(txSig3,Rup);
    for b=1:ndBit3
        if txSigD(b)>0
            if arcvd(b)<0
                c =c+1;
            end
        end
        if txSigD(b)<0
            if arcvd(b)>0
                c =c+1;
            end
        end
    end
    ber1k(a)=c/ndBit3
    c=0;
end

%geração da figura da BER em escala linear, é preciso editar a figura para mudar
%a escala no eixo das ordenadas
figure; plot(EbNo,ber100k,EbNo,ber10k,EbNo,ber1k);

```

3.2 Constelação

No caso da geração dos diagramas de constelação, pode-se ver que na medida que a relação sinal ruído se deteriora, a nuvem de pontos em torno de -1 e +1 se torna mais espalhada. Este comportamento se dá devido à distribuição dos pontos de ruído branco. Como o ruído branco é distribuído de forma gaussiana e aditiva com média 0, cada ponto da constelação será deslocado de um certo valor porém mantendo seu centro, de fato se pudéssemos plotar a distribuição de pontos em torno de -1 e +1 veríamos duas gaussianas.

3.2.1 Sem Ruído

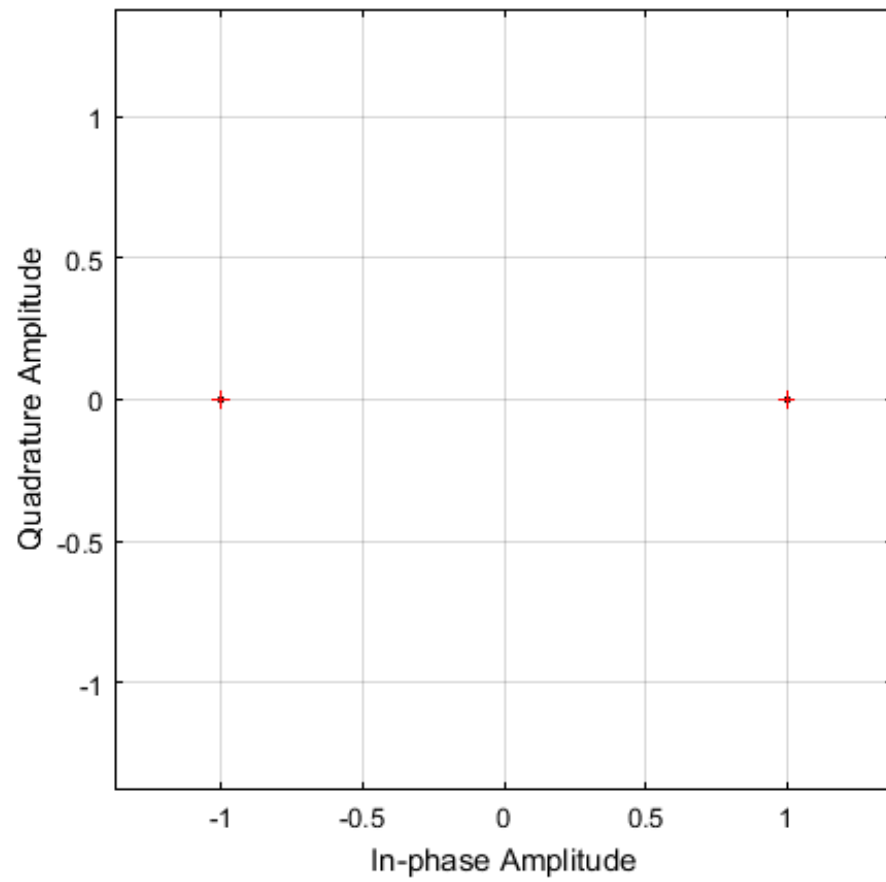


Figura 3.1: Constelação Obtida sem Ruído

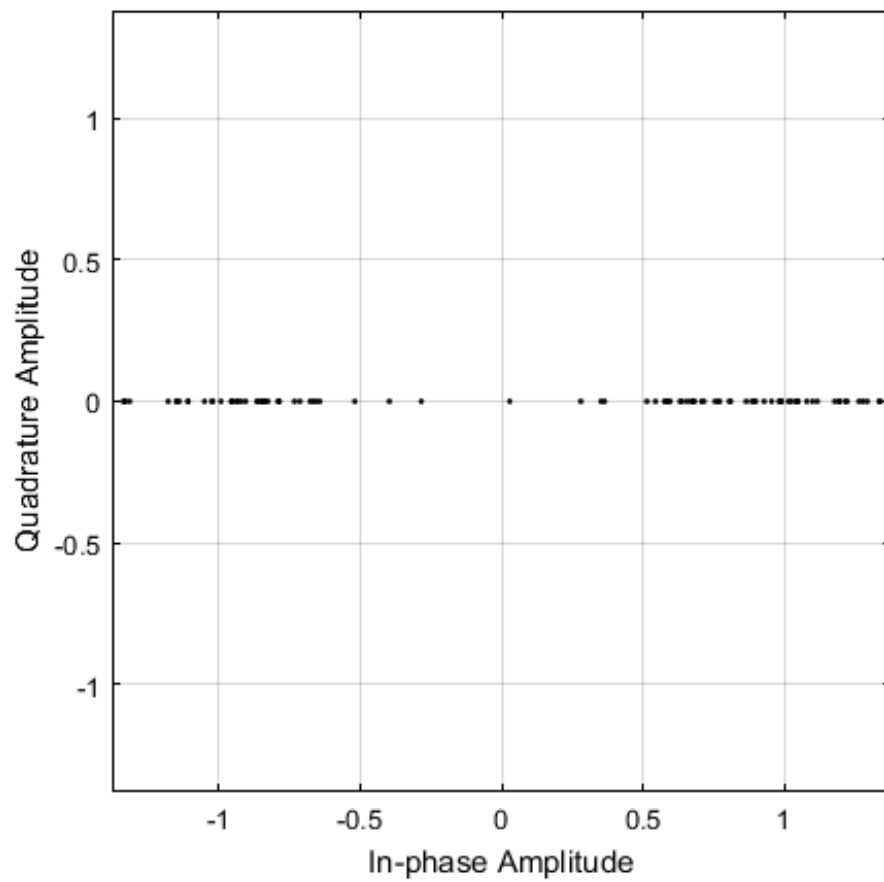


Figura 3.2: Constelação Obtida com $E_b/N_o = 10\text{dB}$

3.2.2 Com Ruído

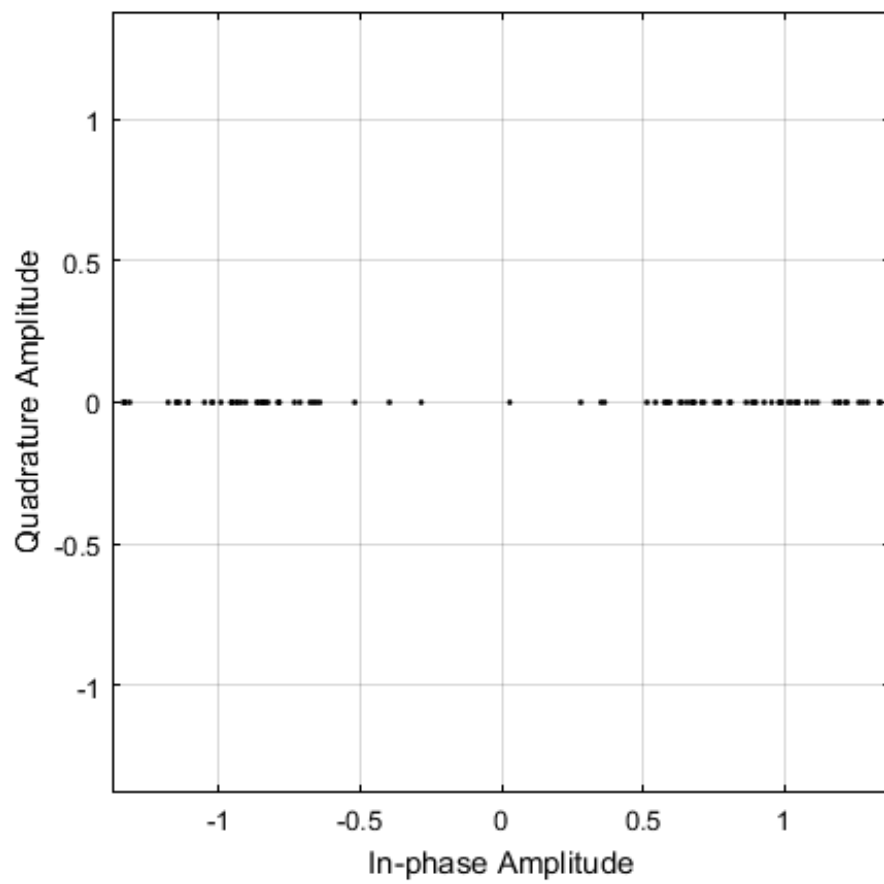


Figura 3.3: Constelação Obtida com $E_b/N_0 = 5\text{dB}$

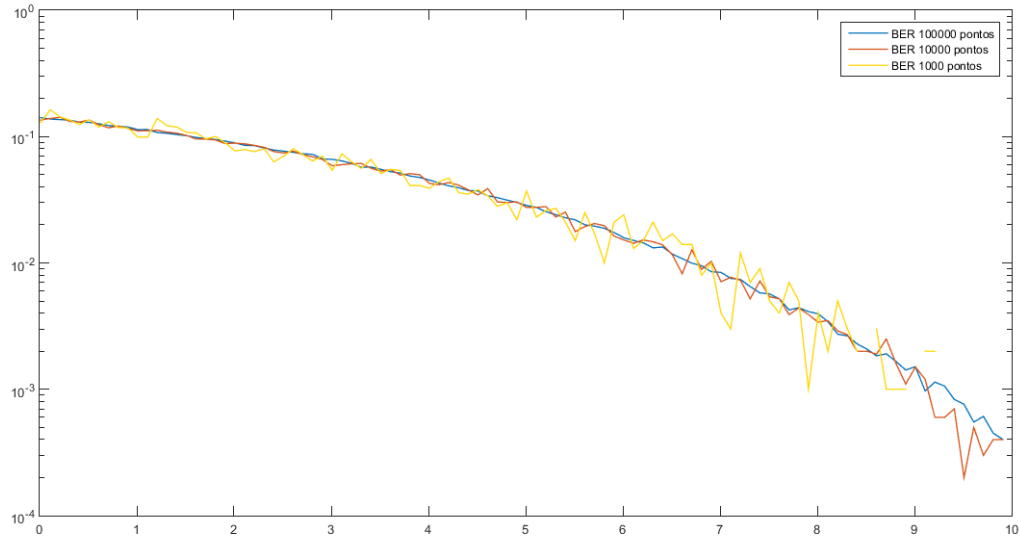


Figura 3.4: Taxas de Erro de Bit para E_b/N_0 em incrementos de 0.1

3.3 Taxa de Erro de Bit e Probabilidade de Erro

É possível ver que para um certo número de bits transmitidos a BER calculada difere um pouco do que seria esperado para a função Q. Este comportamento é esperado pois a natureza estocástica do erro de bit fará com que a medida que aumentamos o número de bits transmitidos usados para calcular a BER, tenhamos um comportamento mais próximo à curva estudada em sala. Em geral temos um pouco mais ou menos erros de bits do que o centro da média que seria a função Q, como transmitimos poucos bits, cada erro terá um peso maior no cálculo da BER, desta forma observamos uma grande flutuação para poucos bits transmitidos e uma flutuação menor para muitos bits pois cada erro individualmente conta menos para o cálculo da BER.

É claramente observável que a medida que melhoramos a relação entre a energia transmitida e o ruído, a BER cai. De acordo com o que estudamos em sala.